CS-302: Artificial Intelligence

LECTURE NO 5:

Engr. Tehseen Ahsan
Assistant Professor, Computer Engineering Department
HITEC University Taxila Cantt, Pakistan

Organization of Lecture

1. Uninformed Search Strategies (***Continuation of Lecture 4***)
    3. Depth-Limited Search Algorithm
    4. Uniform Cost Search Algorithm
    5. Bi-directional Search Algorithm
    6. Iterative Deepening Depth-First Search

3. Depth-Limited Search Algorithm (1/3)

3) Depth-Limited Search Algorithm:

↳ Working is similar to DFS but with a predetermined limit. (to avoid from Infinite loop).

↳ Helps in solving the probelm of DFS.

- DFS: Infinite Path
- Termination Conditions:
  - (i) Failure Value: Probelm/Program has no solution OR solution doesn't exist in given Tree.
  - (ii) Cutoff Failure: Terminates on reaching cutoff value or predetermined depth. ⇒ No Sol?

IMP

3. Depth-Limited Search Algorithm (2/3)

Advantages:

↳ Since we know the depth so less memory is required i.e, it is memory efficient.

Disadvantages:

↳ Can be terminated without finding solution. i.e., in case of 'Cutoff failure'. → Incompleteness

↳ Not optimal. It is not sure whether it will give you a best result.

Time Complexity:

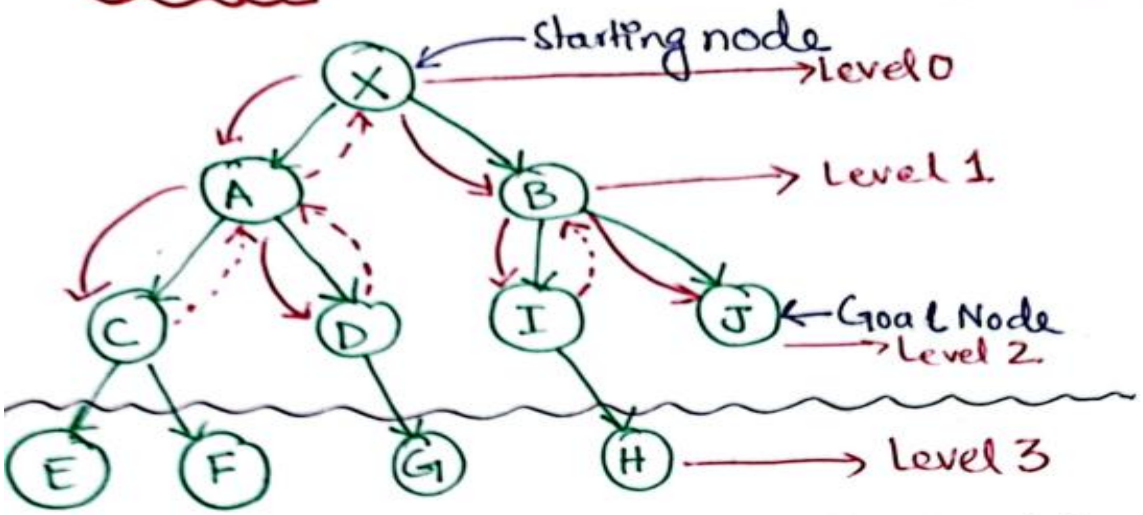
$T.C = O(b^d)$; same as DFS

Space Complexity:

$S.C = O(bd)$; Same as DFS

4. Uniform Cost Search Algorithm (1/3)

Example



→ Let's say we take a predetermined depth as two, i.e.,
- [Depth, d = 2]

→ Traversal path

X → A → C → D → B → I — J

→ Let's say if we were given Goal node as 'H' with predetermined depth as 2. This depth will result in no solution. Because the solution is available at depth, d = 3. ⇒ Cutoff Failure Termination Condition.

4. Uniform Cost Search Algorithm (1/5)

4) Uniform Cost Search Algorithm

↳ Used for weighted Tree/Graph Traversal.

↳ Goal is to path finding to goal-node with lowest cummulative cost i.e., finding optimal path.

↳ Node expansion is based on path costs.

↳ Priority Queue is used for implementation.
- High priority to minimum cost path.

↳ Back Tracking can also be implemented in this algorithm.

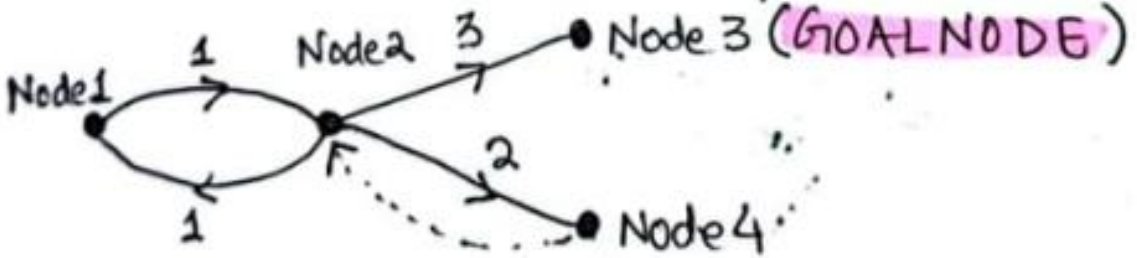
4. Uniform Cost Search Algorithm (2/5)

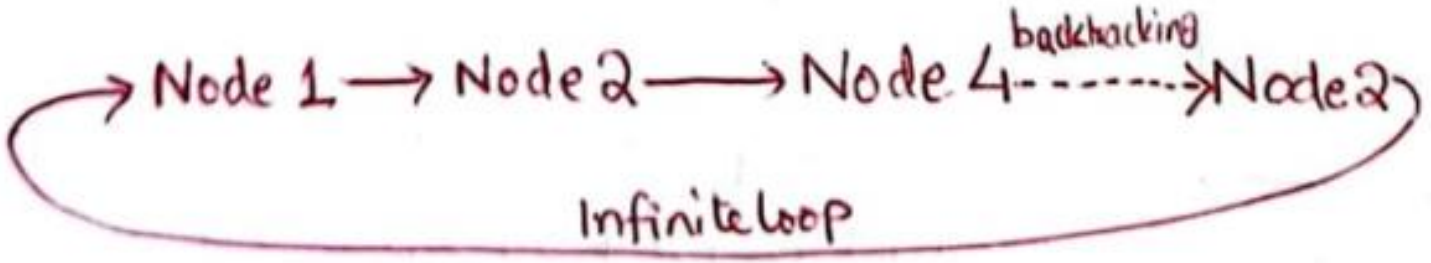
Advantages:

↳ It provides optimal solution. (Not Always).

Disadvantages:

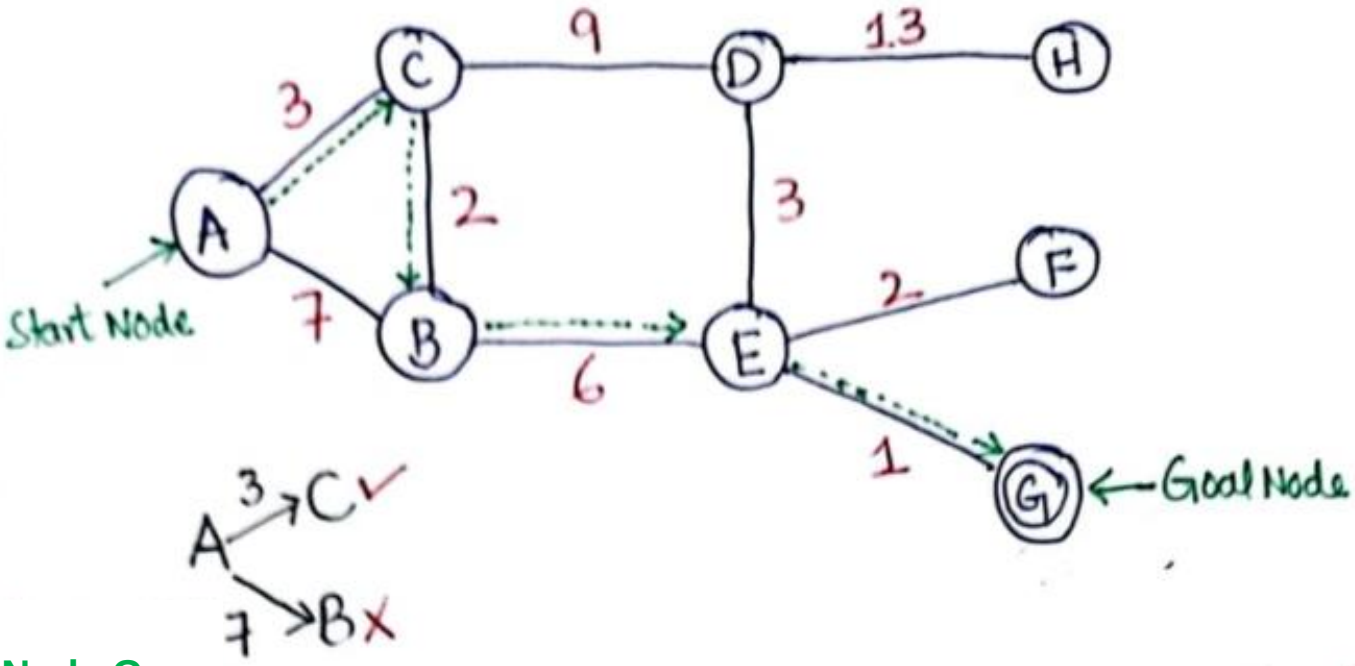
↳ It can struck in Infinite loop.





4. Uniform Cost Search Algorithm (3/5)

Example



A → 3 → C ✓

A → 7 → B ✗

Goal Node G:

↳ Path to Goal Node (G) ⟹ Backtracking is not involved.

A $\xrightarrow{3}$ C $\xrightarrow{2}$ B $\xrightarrow{6}$ E $\xrightarrow{1}$ G

Total Cost = 3+2+6+1 = 12

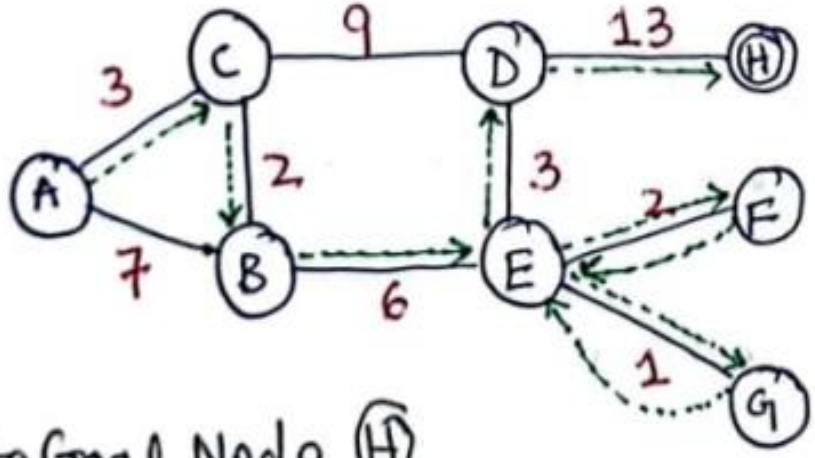
4. Uniform Cost Search Algorithm (4/5)

Goal Node H:

↳ Suppose Now select the Goal node as 'H'

↳ Backtracking will be involved.

↳ Redrawing above example:



Path to Goal Node (H)

$A \xrightarrow{3} C \xrightarrow{2} B \xrightarrow{6} E \xrightarrow{1} G \xrightarrow{1} E \xrightarrow{2} F \xrightarrow{2} E \xrightarrow{3} D \xrightarrow{13} H$

Total Cost = 3+2+6+1+1+2+2+3+13 = 33

4. Uniform Cost Search Algorithm (5/5)

→ Sometimes we don't get optimal solution, this algorithm tries to find the optimal solution. For example for Goal Node 'H', the optimum path could be:

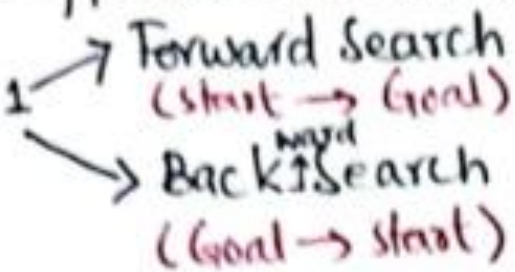
$$A \xrightarrow{3} C \xrightarrow{9} D \xrightarrow{13} H$$

Total Cost = $3+9+13 = 25 < 33$

5. Bi-directional Search Algorithm (1/4)

5) Bidirectional Search Algorithm

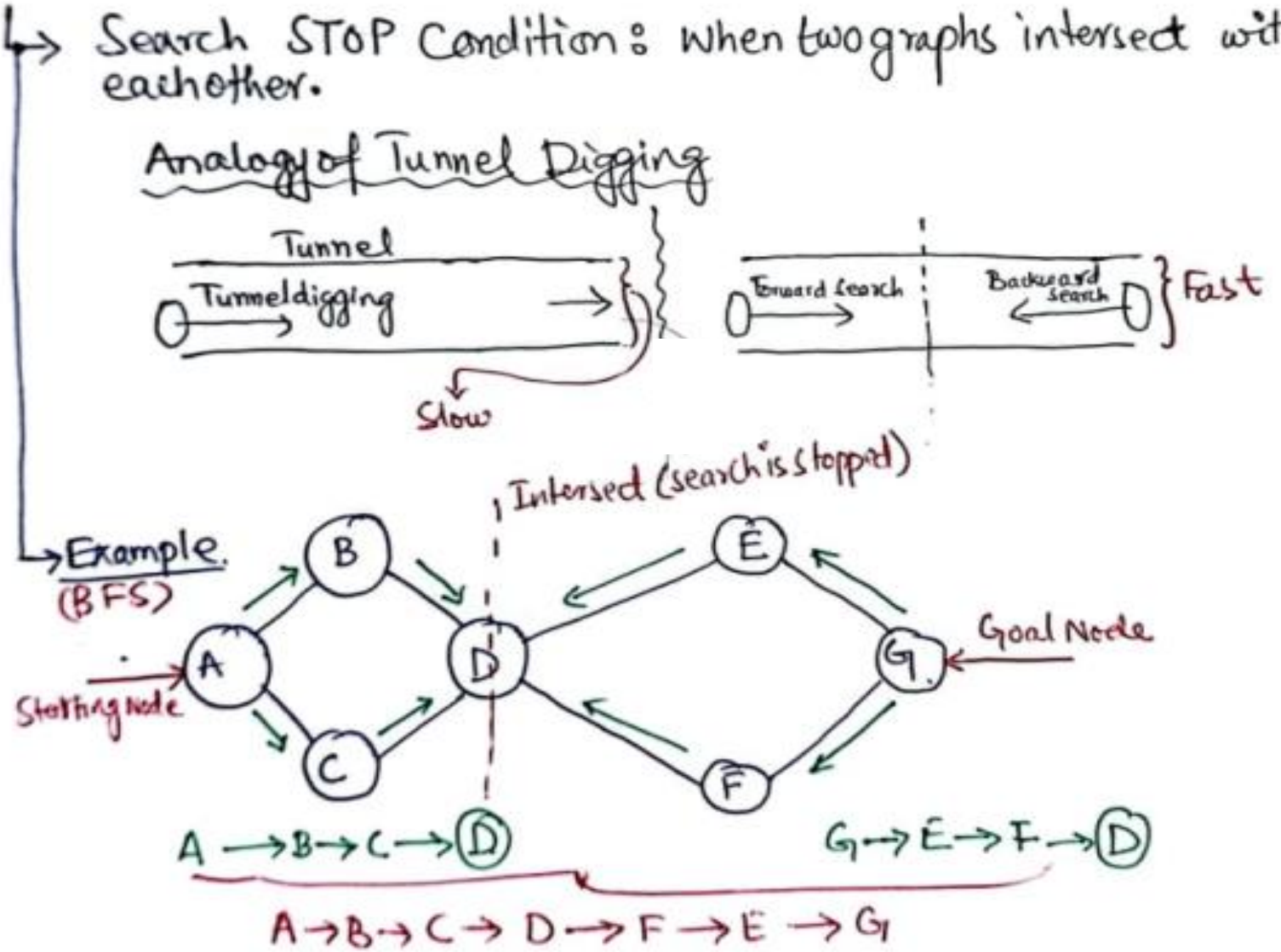
↳ Two different searches run simultaneously



↳ Single Search Graph is replaced with two small graphs.

↳ Any search technique can be used. (BFS, DFS, ...)

5. Bi-directional Search Algorithm (2/4)



5. Bi-directional Search Algorithm (3/4)

Advantages:

i) Fast searching technique.

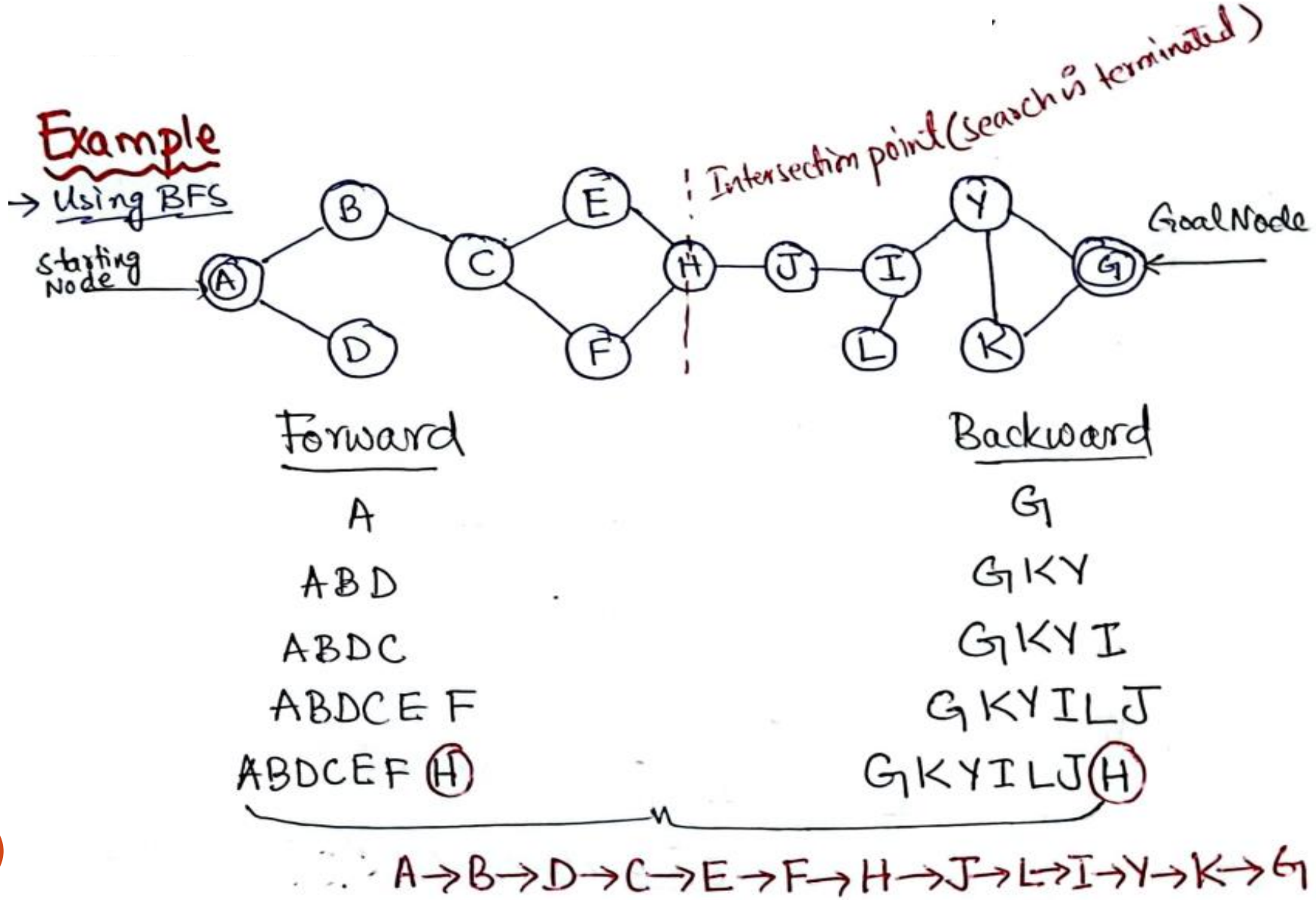
ii) Requires less memory.

Disadvantages:

i) Implementation is difficult. (two different data structures are maintained).

ii) Goal state should be known in advance.

5. Bi-directional Search Algorithm (4/4)



6. Iterative Deepening Depth-First Search (1/2)

6) Iterative Deepening Depth-First Search:

↳ Combination of both DFS and BFS.

↳ Best Depth limit is found out by gradually increasing limit. Initial depth limit is '0'. On every iteration increase depth by 1.

↳ Stack is used for DFS part.

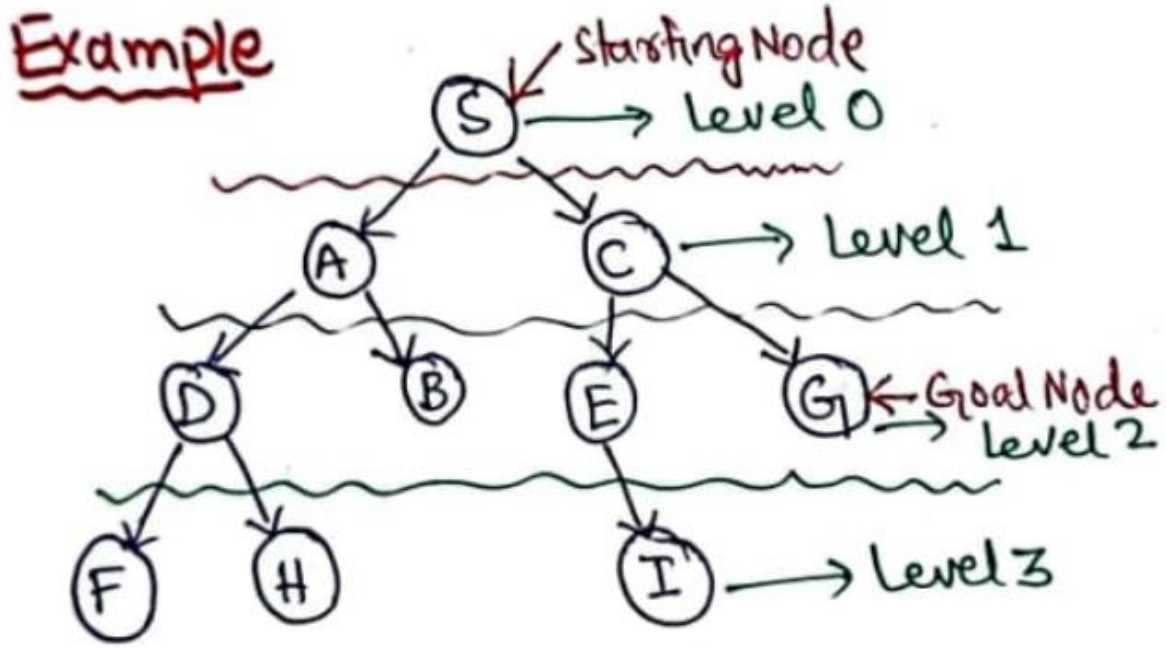
Advantage:

↳ Includes benefits of both BFS & DFS. {Fast Searching or less memory requirements}.

Disadvantage:

↳ Repeats the work of previous phase. in each iteration.

6. Iterated Deepening Depth-First Search (2/2)



* 1st Iteration, d=0 [S]
* 2nd Iteration, d = 0+1 = 1 [S → A → C]
* 3rd Iteration, d = 1+1 = 2 [S → A → D → B → C → E → G]

References: Textbooks

1. *"Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill companies Inc., Chapter 1-22 page 1-613.*
2. *"Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig, (2010), Prentice Hall, Chapter 1-27. page 1-1151.*
3. *"Computational Intelligence: A logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 1-12, page 1-608.*
4. *"Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F.Lugar, (2002), Addison-Wesley, Chapter 1-16, page 1-743.*
5. *"AI: A new Synthesis", by Nils J.Nilsson, (1998), Morgan Kaufmann Inc., Chapter 1-25, Page 1-493.*
6. *"Artificial Intelligence: Theory and Practice" by Thomas Dean, (1994), Addison-Wesley, Chapter 1-10, Page 1-650.*
7. *Related documents from open source, mainly internet.*